C - PROGRAM STRUCTURE

It illustrates[Describes] how to write a program in c-language.

Every programming language is having a particular structure and we should have to follow this structure.

C-Programming structure is divided into the following parts.

- **[documentation section]**
- **Header files / Proto types / Preprocessor**
- **[global variables]**
- **[function declarations & definitions]**
- **void main() / main() / int main()**
- **Other statements.**

Generally documentation section consists of program headings, definitions etc and They should be represented with comments.

The statements that are enclosed in between
/* and */ are called comments.

Comments never participate in program execution. They are only for user understandability or display purpose.

C-Language supports comment block only.

Eg:

```
/*
………;
………;
*/
```

C++ supports comment block and single line comments.

Eg: // ……………………

Header files consists of function definitions, global variables, macros etc.

We can declare the header files at any place of our program. But before going to use the relevant function, its header file should be declared. It is recommended to declare the header files at the top of the program.

Every header file should be started with **#include**. Here **#** is a **preprocessor** indicator.

We can place header files in angled brackets

< > or double quotes **“ “**.

Header file never ends with **semicolon**(**;**).

Note: In C^{++}, we should have to declare header files at the top only.

The variables that are declared before main() or top of the program are called **global variables** and they can be accessed from anywhere in our program. They are optional.

Function declarations and definitions contain function header and body.

* Every C-Program execution starts from main() function and travel towards down. Hence it is also called **top-down** approach.

* Without main(), C-Program never executed but compiled.

* main() is predefined function with user defined body. main() doesn't have any header file. One program have to maintain one main() only. We can create alternate for main(). Other statements are changed from program to program.

Note: It is recommended to write C programs in lower case only. Every statement should have to end with semicolon main().

printf():

It is a predefined function available in stdio.h

It is a the major output function in c.
it is used to display the given text on the monitor.

In printf, f means formatted.

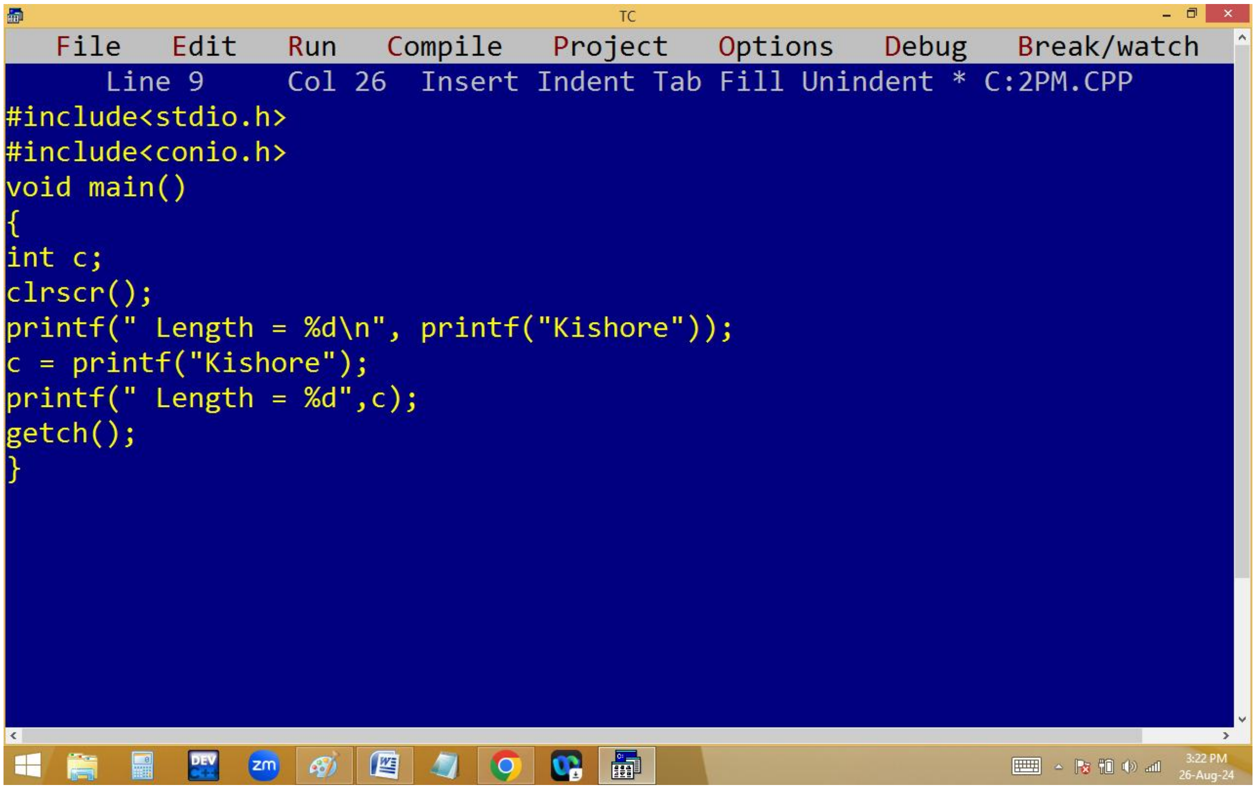
Syntax:

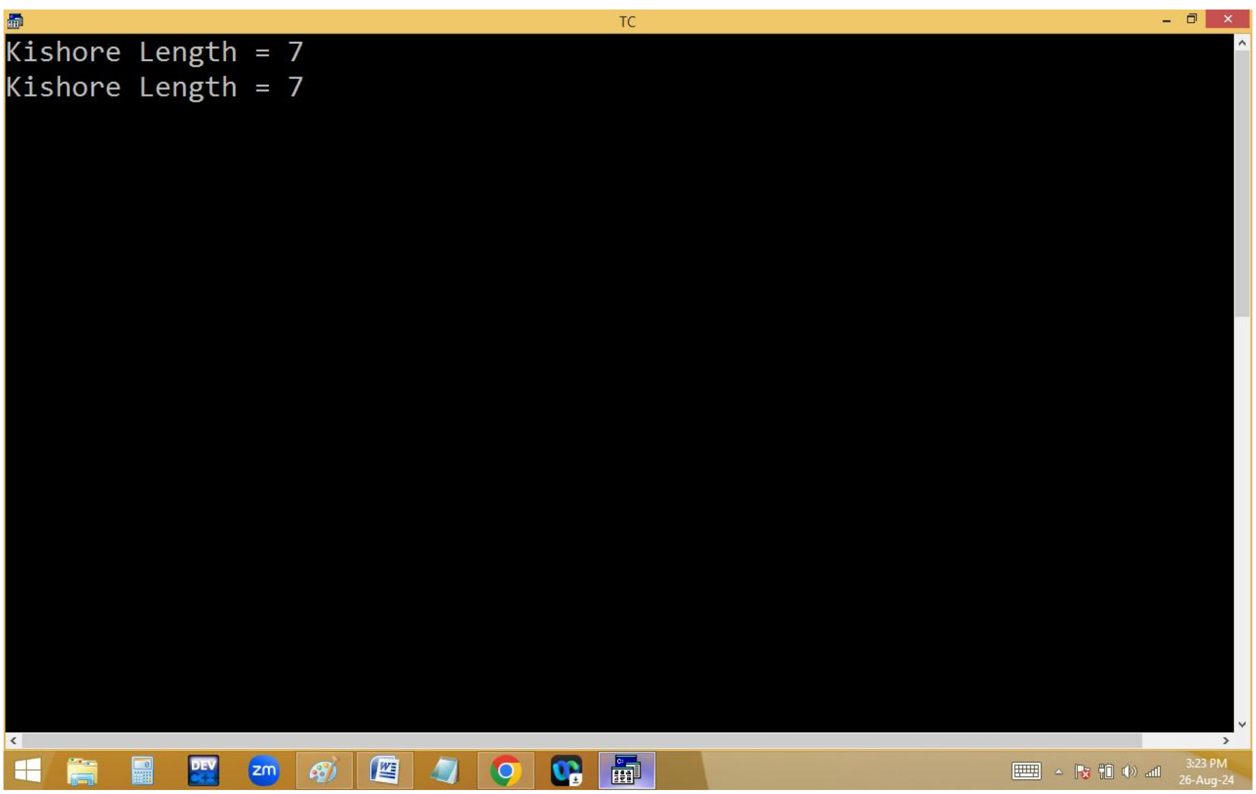
int printf("[text] [conversion characters / format specifiers / format strings] " [, variables] [, expressions]);

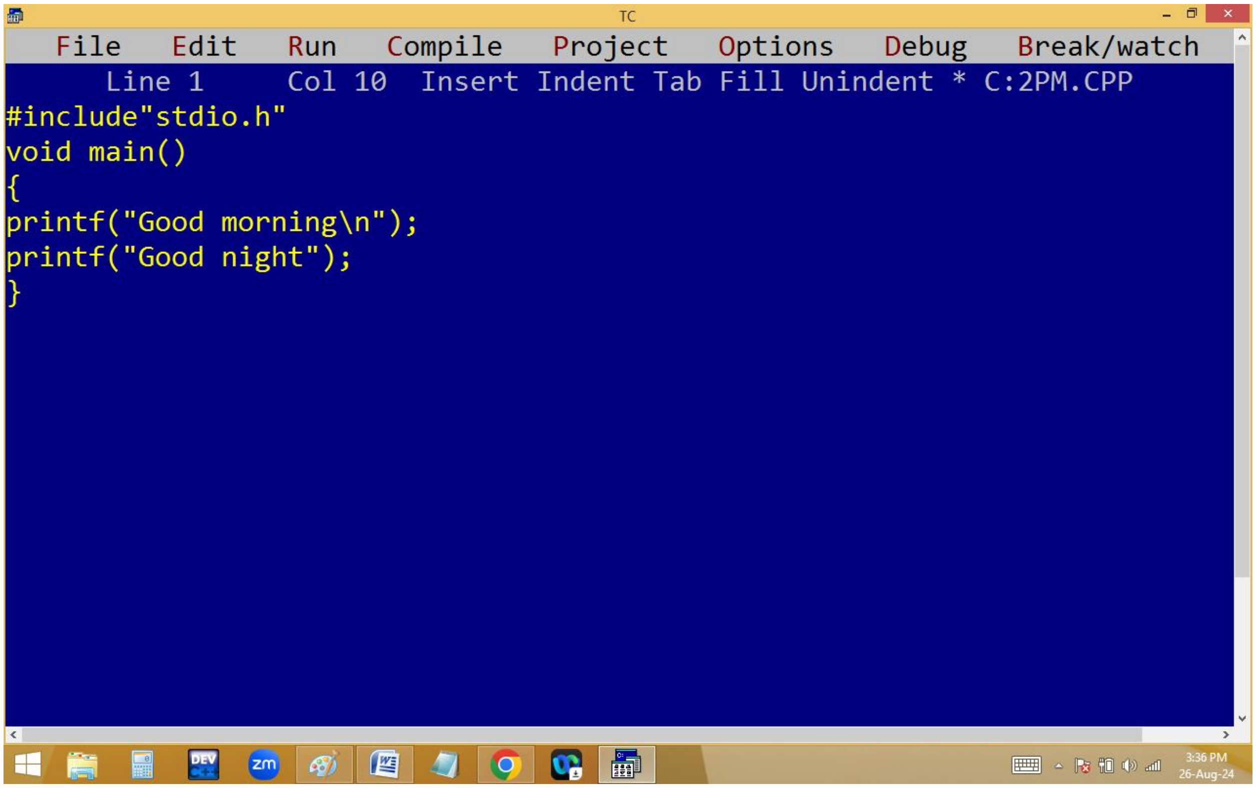
note:

1. In printf the first argument / value should be in " ".
2. In printf execution order is right to left but printing left to right.
3. In printf everything printed as it is except back slash and conversion characters.
4. Printf can perform both formatted and unformatted outputs.
   eg:
   printf("hi"); 🡸 unformatted
   printf("%d",10);🡸 formatted
5. Printf always return int that indicate the no of printable characters.

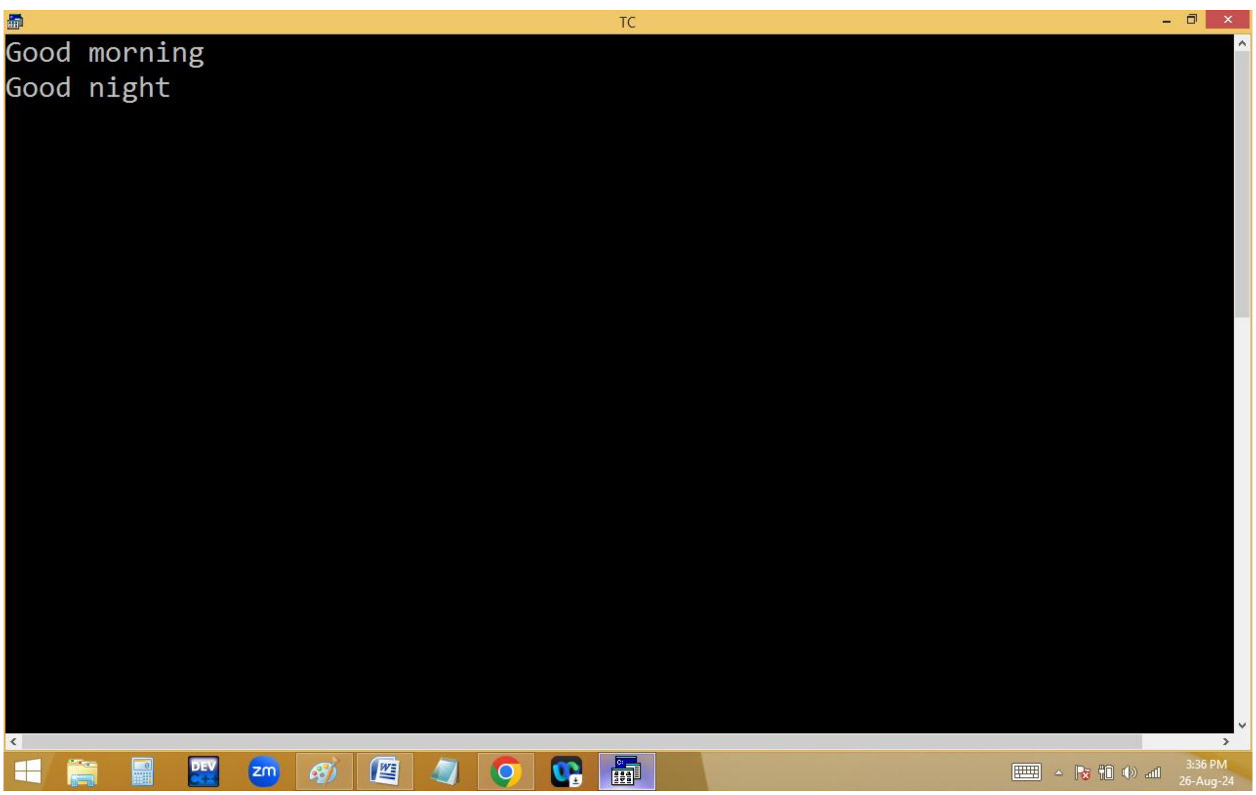
write a c program to find string length without using strlen() or a loop.


TC
File Edit Run Compile Project Options Debug Break/watch
Line 9 Col 26 Insert Indent Tab Fill Unindent * C:2PM.CPP
#include<stdio.h>
#include<conio.h>
void main()
{
int c;
clrscr();
printf(" Length = %d\n", printf("Kishore"));
c = printf("Kishore");
printf(" Length = %d",c);
getch();
}
3:22 PM
26-Aug-24

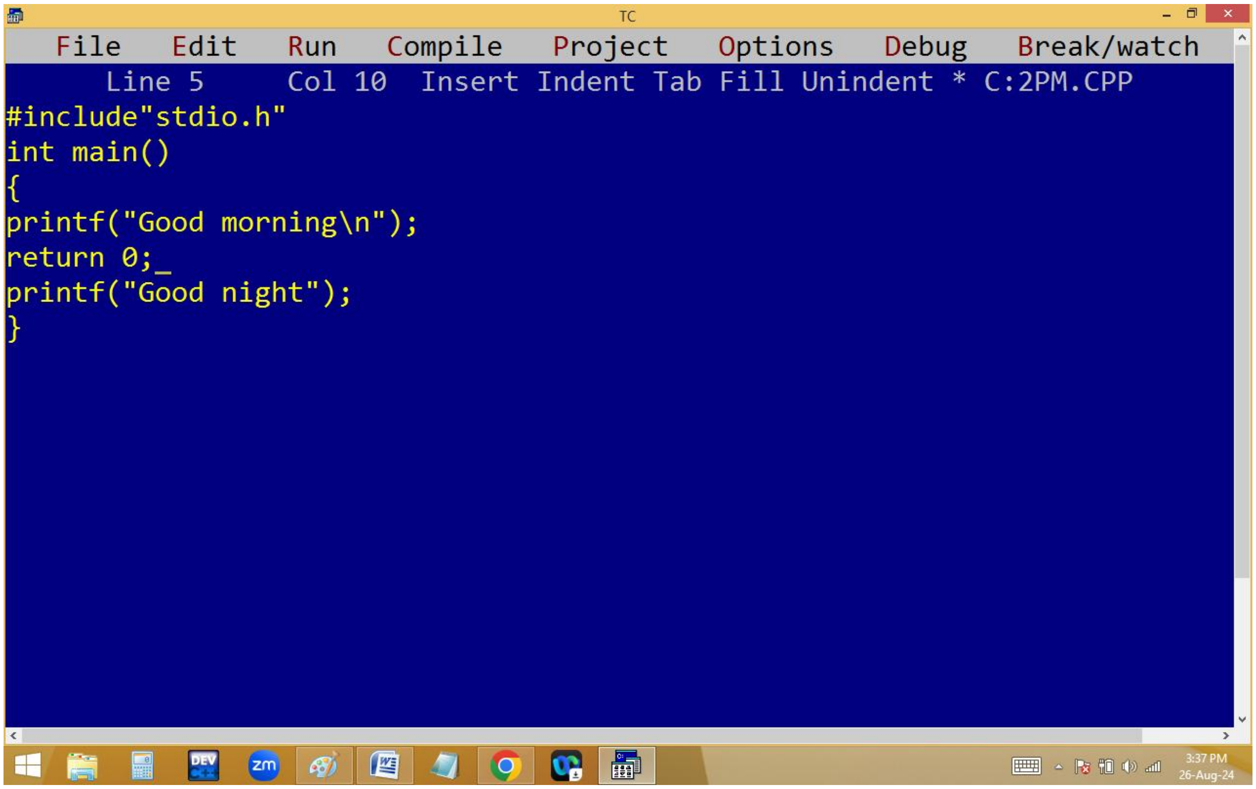

TC
Kishore Length = 7
Kishore Length = 7
3:23 PM
26-Aug-24

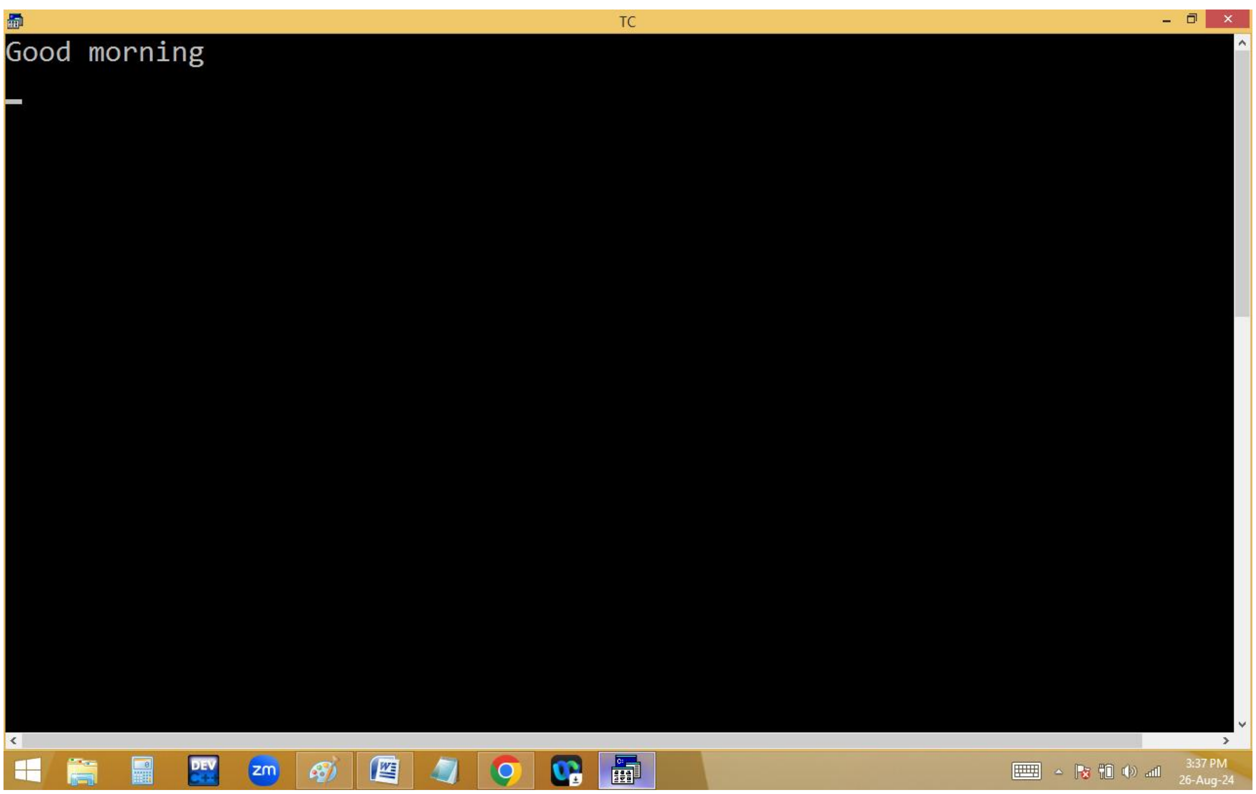


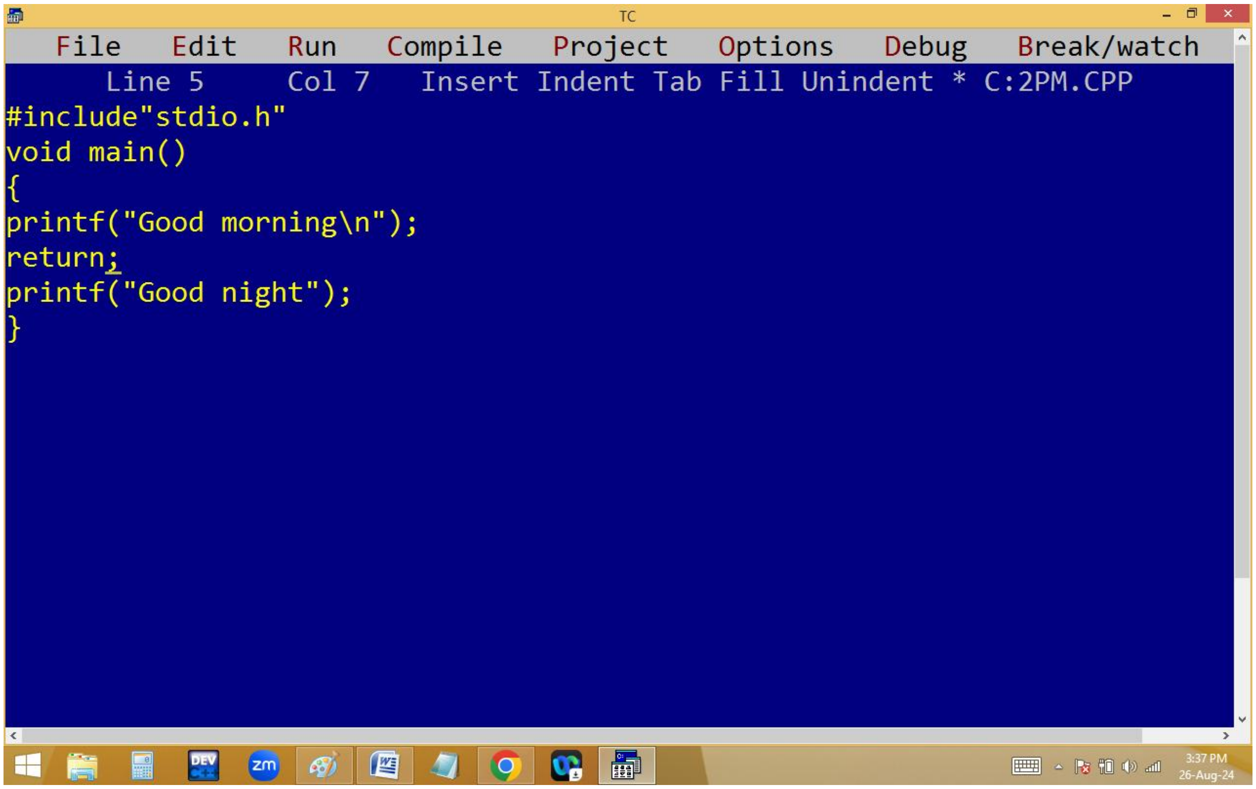

```
#include"stdio.h"
void main()
{
printf("Good morning\n");
printf("Good night");
}
```

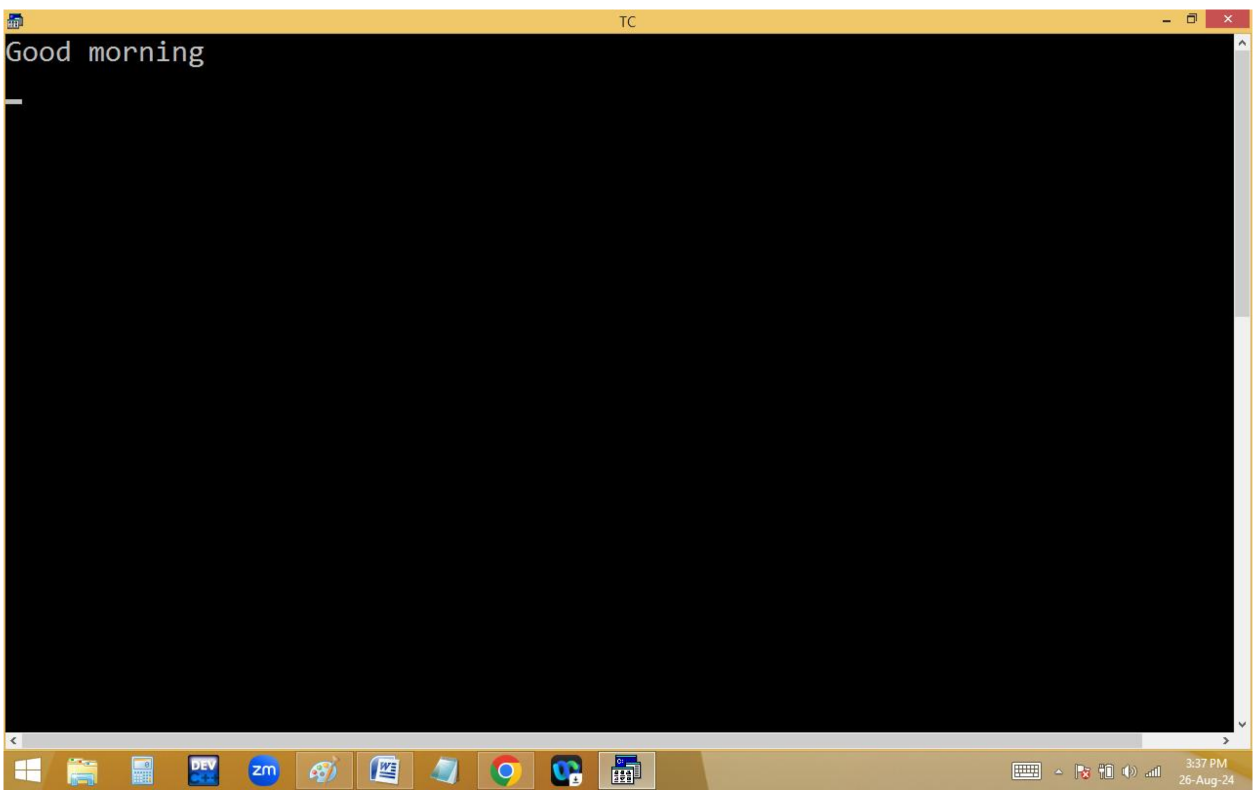


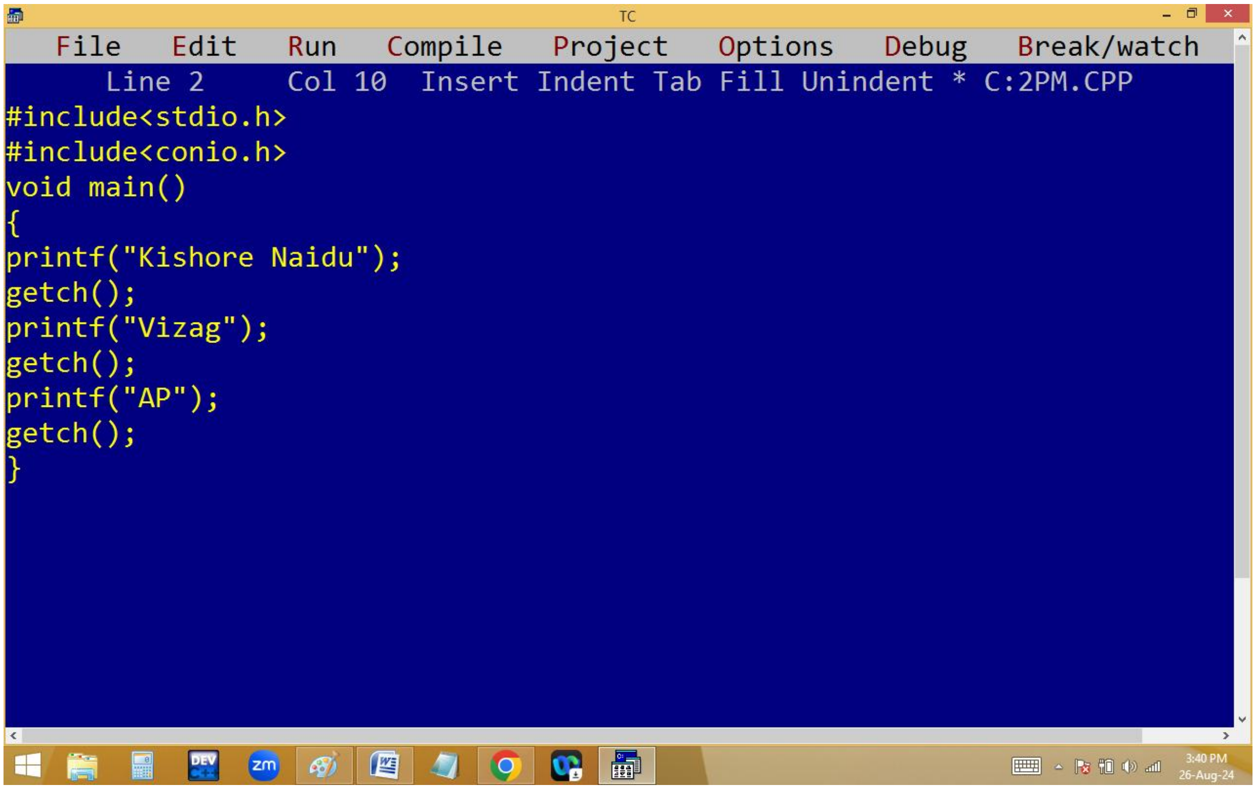

```
Good morning
Good night
```

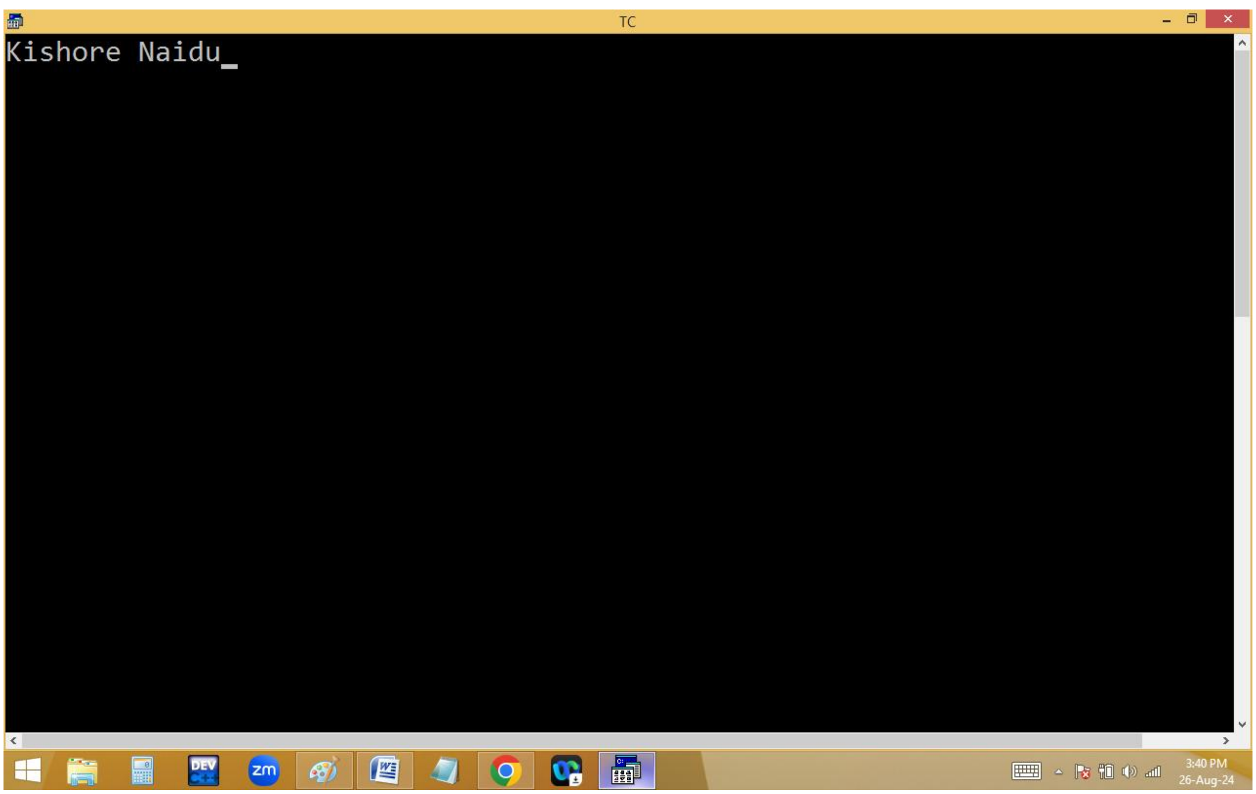

TC
File Edit Run Compile Project Options Debug Break/watch
Line 5 Col 10 Insert Indent Tab Fill Unindent * C:2PM.CPP
#include"stdio.h"
int main()
{
printf("Good morning\n");
return 0;_
printf("Good night");
}
3:37 PM
26-Aug-24

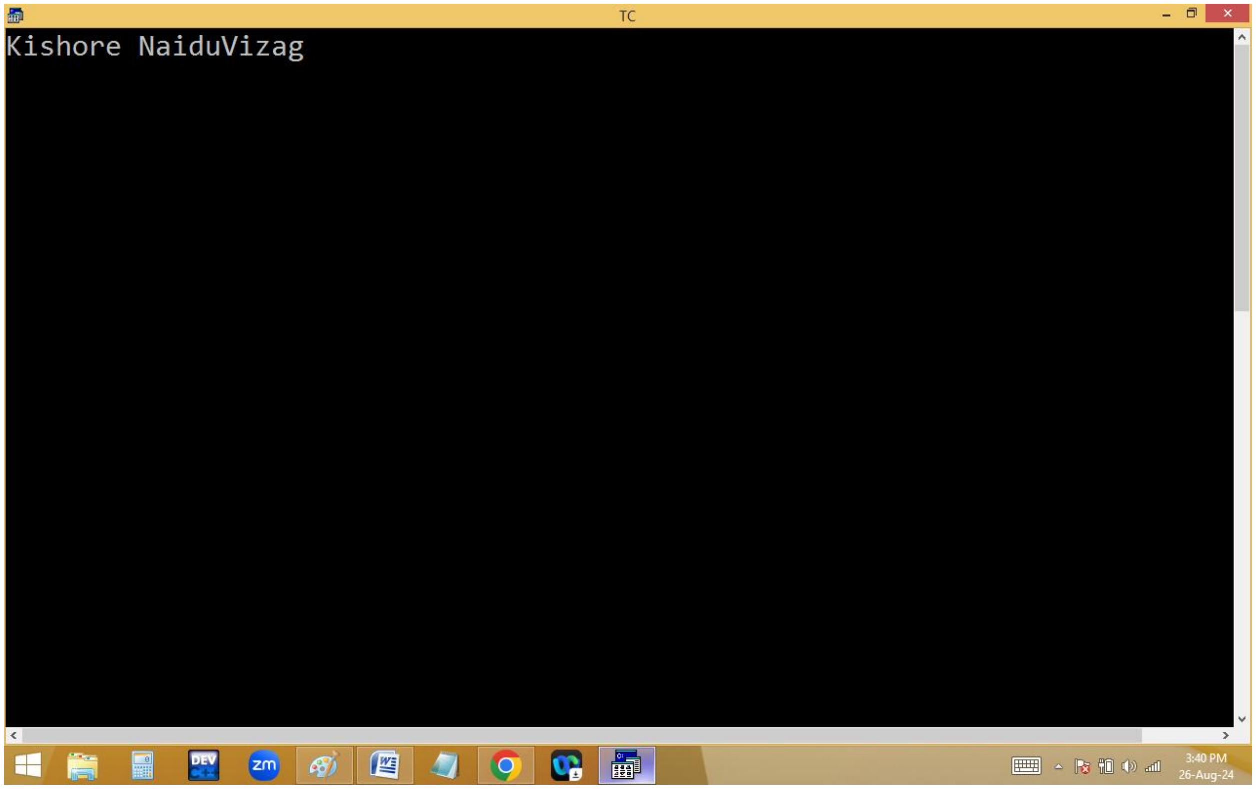

TC
Good morning
3:37 PM
26-Aug-24

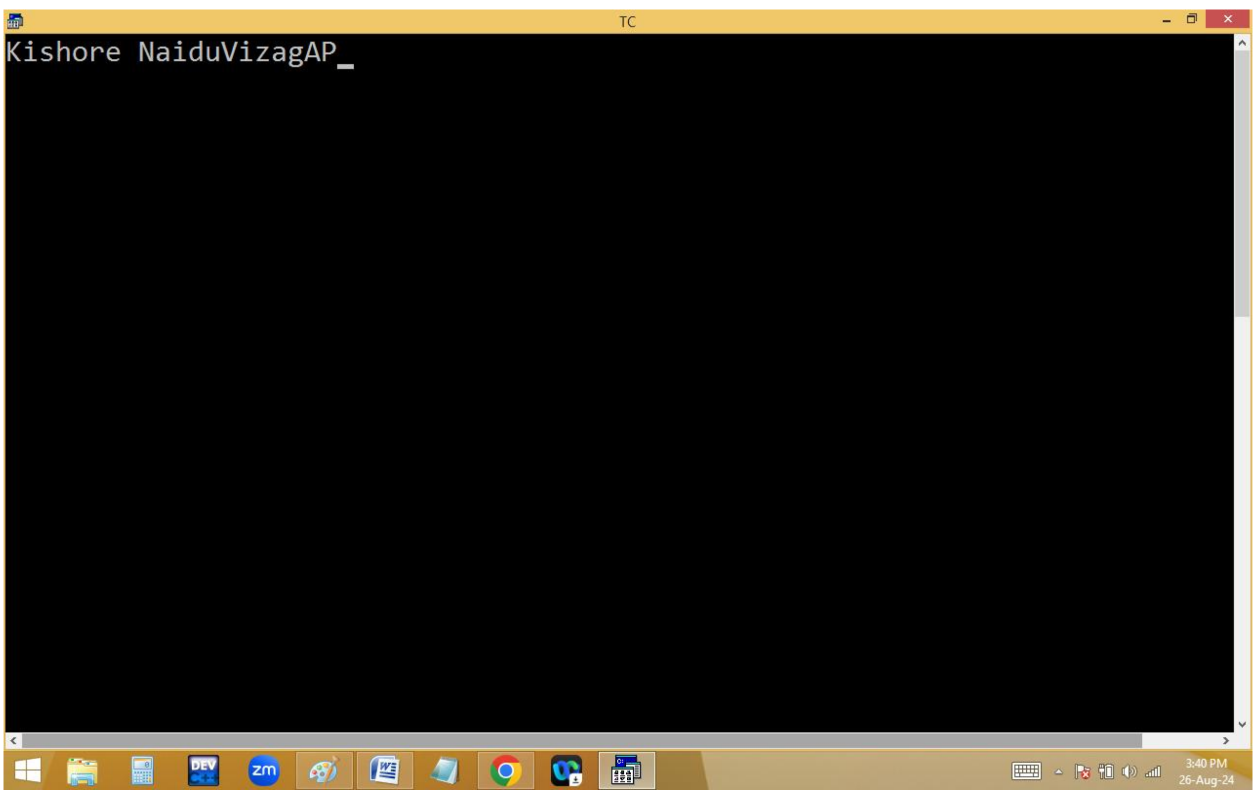

TC
File Edit Run Compile Project Options Debug Break/watch
Line 5 Col 7 Insert Indent Tab Fill Unindent * C:2PM.CPP
#include"stdio.h"
void main()
{
printf("Good morning\n");
return;
printf("Good night");
}
3:37 PM
26-Aug-24

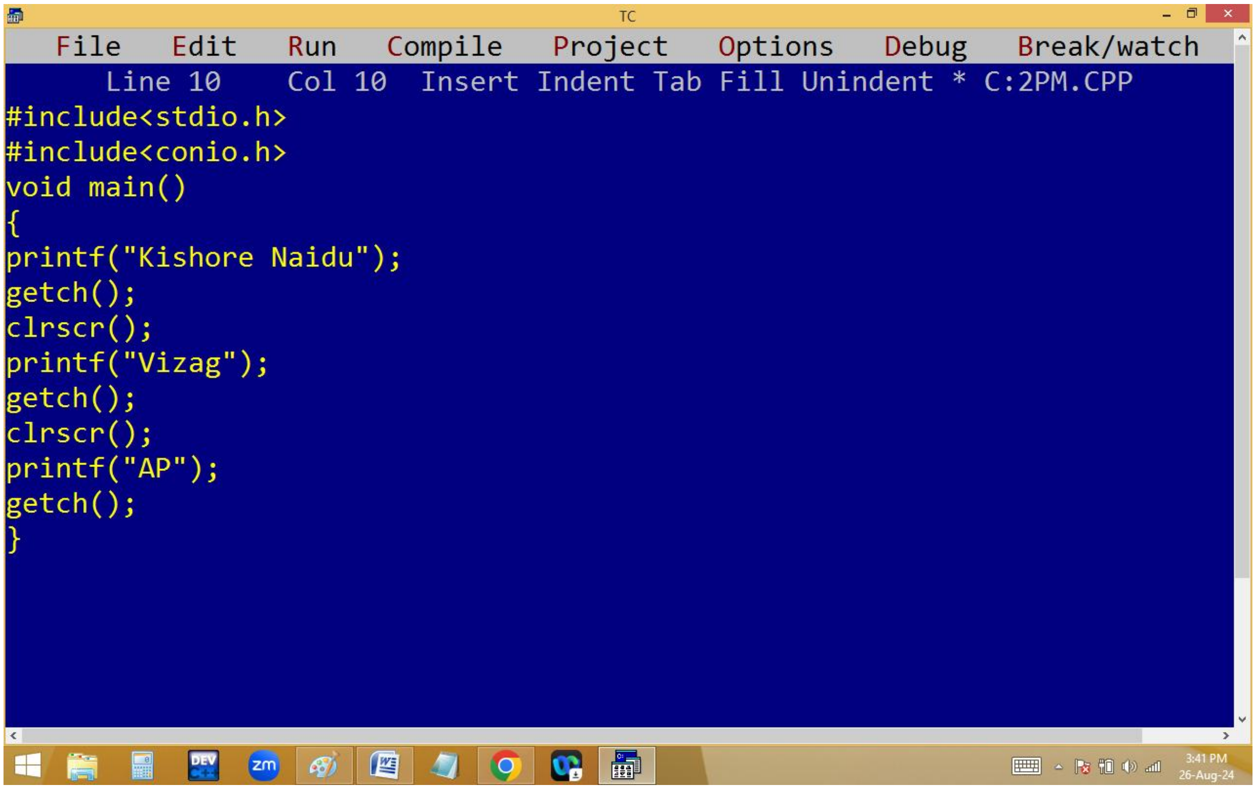

TC
Good morning
3:37 PM
26-Aug-24


TC
File Edit Run Compile Project Options Debug Break/watch
Line 2 Col 10 Insert Indent Tab Fill Unindent * C:2PM.CPP
#include<stdio.h>
#include<conio.h>
void main()
{
printf("Kishore Naidu");
getch();
printf("Vizag");
getch();
printf("AP");
getch();
}
3:40 PM
26-Aug-24

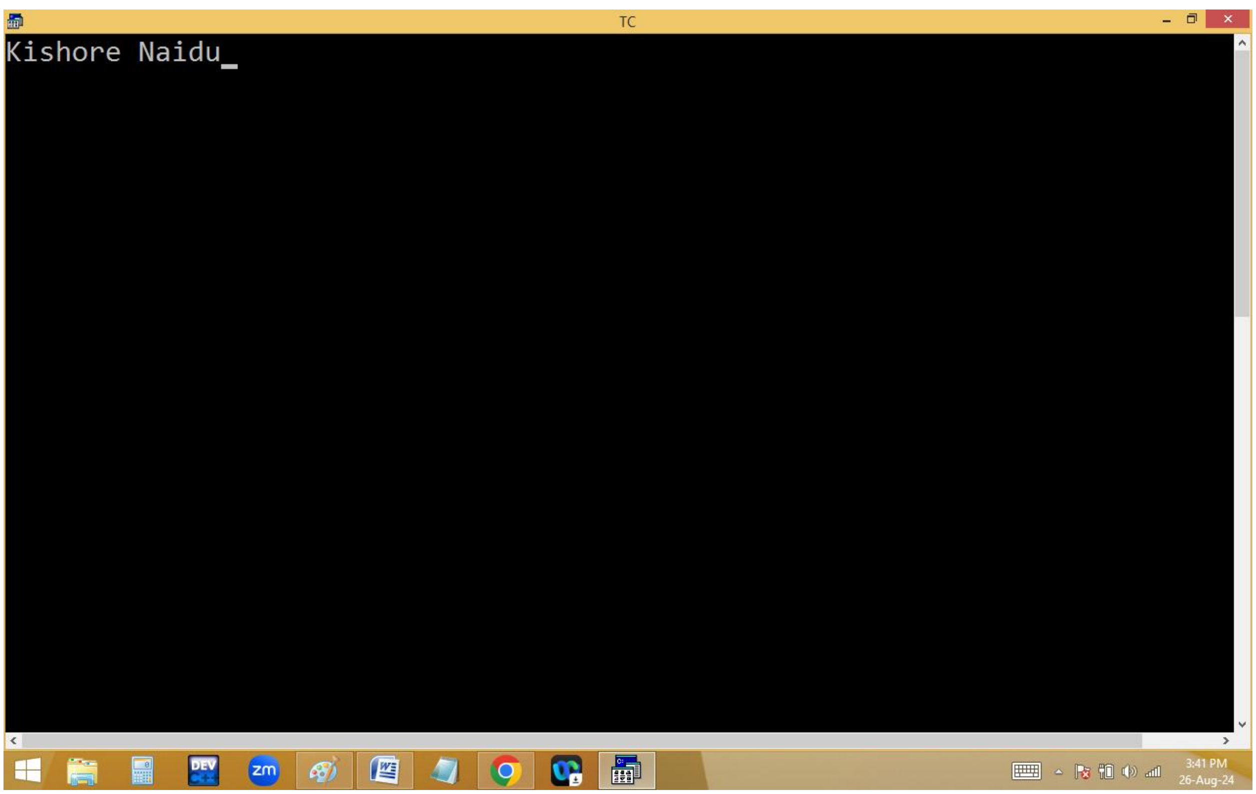

TC
Kishore Naidu_
3:40 PM
26-Aug-24

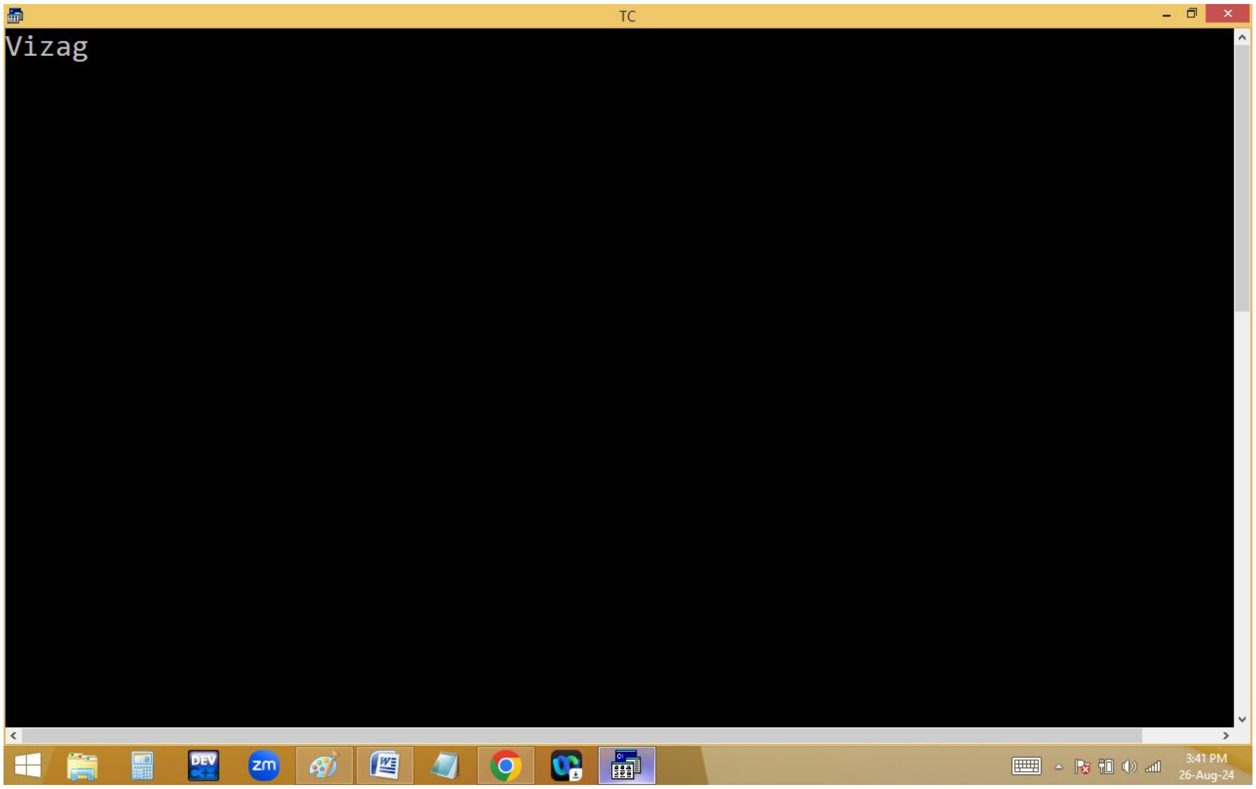

TC
Kishore NaiduVizag
3:40 PM
26-Aug-24

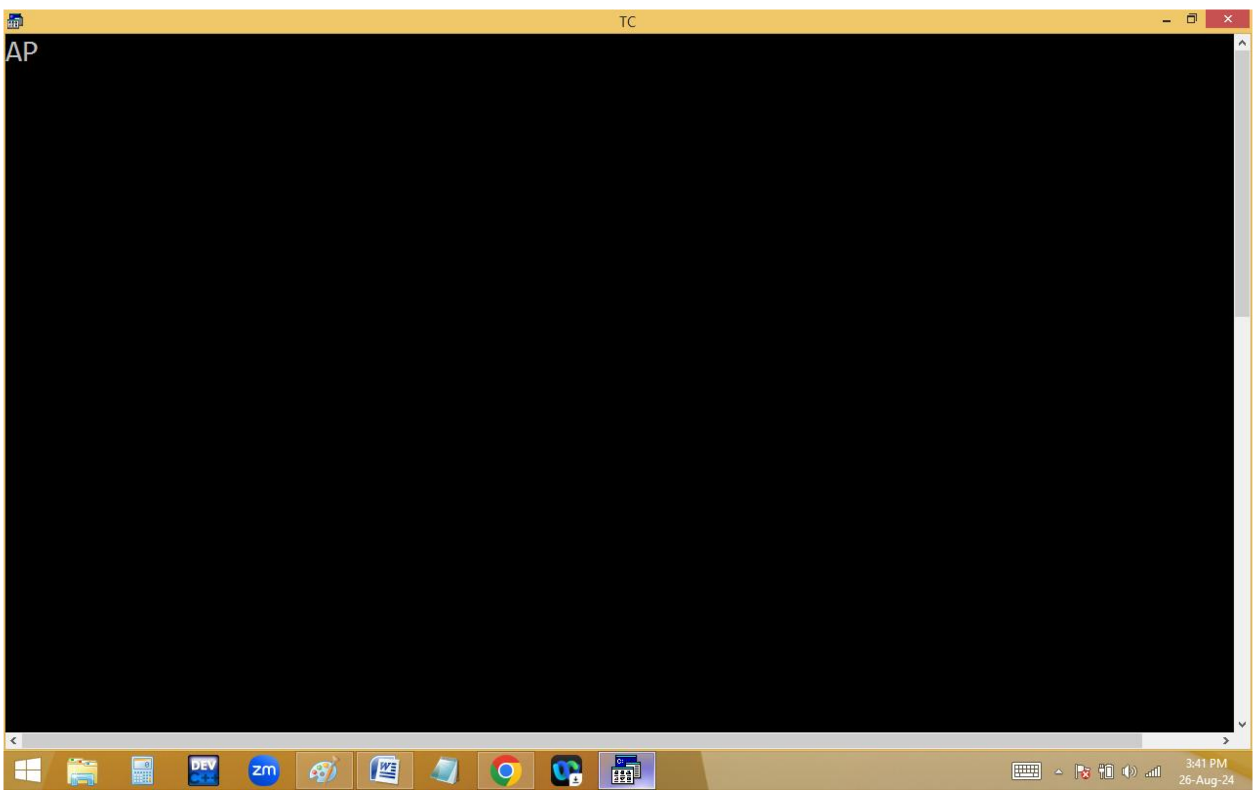

TC
Kishore NaiduVizagAP_
3:40 PM
26-Aug-24




```
#include<stdio.h>
#include<conio.h>
void main()
{
printf("Kishore Naidu");
getch();
clrscr();
printf("Vizag");
getch();
clrscr();
printf("AP");
getch();
}
```

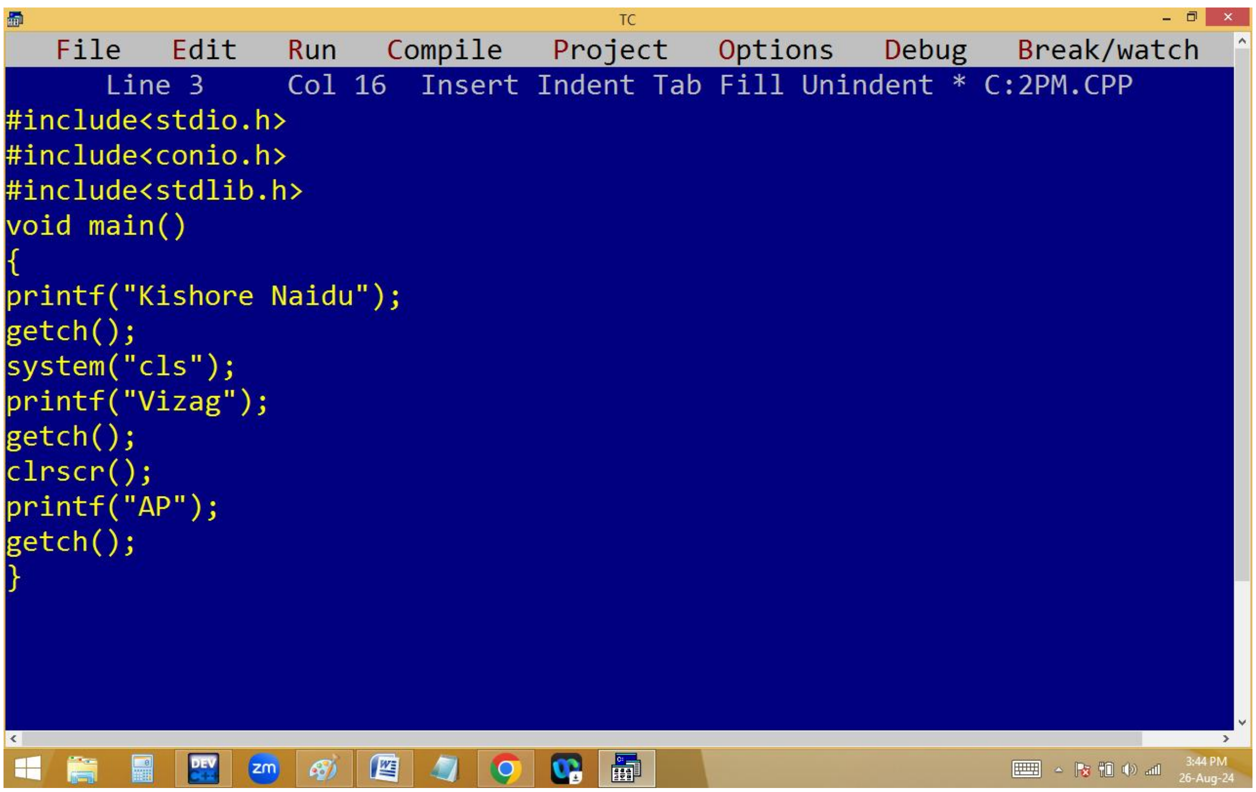







```
#include<stdio.h>
```

```
#include<conio.h>

#include<stdlib.h>

main()

{

	printf("Kishore");

	getch();

	system("cls");

	printf("Naidu");

	getch();

	system("cls");

}
```



<u>BACK SLASH / ESCAPE SEQUENCE CHARACTERS</u>

They started with back slash [\ ].

They used to format the outputs.

They participated in program execution but not displayed in output. Hence they are also called **escape sequence characters**.

Each back slash character=**1 byte i.e. one character.**

BACK SLASH CHARACTER	DESCRIPTION
\a	Alert [beep sound]
\b	Back space
\n	New line character
\t	Tab space
\r	Carriage return[beginning of line]
\f	Form feed ♀
\v	Vertical tab ♂
\0	Null char
\\	\ [invalid]
\k	k [invalid]